

Learning with Projections

Feasibility based optimization for Neural Networks

Promotor: Panos Patrinos
Daily Supervisor: Jan Quan

Jeff Jambé

Agenda

01 Introduction

- Constraints in neural networks
- Projections of common constraints
- Solving feasibility problems

02 Experiments

03 Planning

Introduction

Paradigm Shift

Gradient Based

Minimize a loss function:

$$\min_{\theta} \mathcal{L}(f(x; \theta), y)$$

Update step (Gradient Descent):

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Feasibility Based

Find a point in the intersection of constraints:

$$\theta \in \bigcap_{i=1}^m C_i$$

Update step (e.g. Cyclic Projections):

$$\theta \leftarrow P_{C_1} P_{C_2} \dots P_{C_m}(\theta)$$

Motivation

- ▶ **Gradient free**

Removes need for differentiable activation functions, enabling novel architectures (e.g. gapped step activation, ...).

- ▶ **High degrees of parallelization**

Projections can often be computed in parallel, enabling efficient large-scale training.

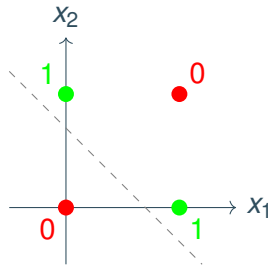
- ▶ **Feasible in the overparametrized case**

Running Example: Binary XOR Classification

Problem Definition

The XOR Problem

- **Input:** $x_{\text{data}} \in \{0, 1\}^2$
- **Output:** $y \in \{0, 1\}$ (1 iff $x_1 \neq x_2$)
- **Datapoints:** $k = 4$
- **Challenge:** Not linearly separable.
- **Requirement:** Needs at least one hidden layer.



Methodology

Constraints for the XOR Classifier

1. Data Constraints:

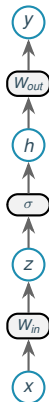
$$\begin{cases} \forall k, c[k] = 1 : y[k] \geq \delta \\ \forall k, c[k] = 0 : y[k] \leq 0 \end{cases} \quad \forall k : x[k] = x_{\text{data}}[k]$$

2. Consensus Constraints:

$$\forall k : W_{in}[k] = W_{in}, \quad W_{out}[k] = W_{out}$$

3. Model Constraints:

$$\forall k : z[k] = W_{in}[k] \begin{bmatrix} x[k] \\ 1 \end{bmatrix}; h[k] = \sigma(z[k]) = \begin{cases} 1 & \text{if } z[k] \geq 0 \\ 0 & \text{otherwise} \end{cases}; y[k] = W_{out}[k] \begin{bmatrix} h[k] \\ 1 \end{bmatrix}$$



Solving Feasibility Problems

Find $x \in \bigcap C_i$.

Projection on to set: $P_C(x) = \arg \min_{y \in C} \|x - y\|$

Strong theoretical results in the convex case; but not in the non-convex case.

Common Algorithms:

Alternating Projections (AP)

$$x_{k+1} = P_{C_2}(P_{C_1}(x_k))$$

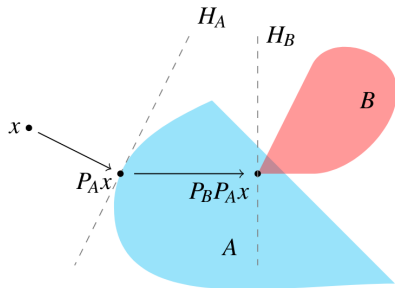
Douglas-Rachford (DR)

$$\begin{aligned} R_C &= 2P_C - I \\ x_{k+1} &= \frac{1}{2}(I + R_{C_2}R_{C_1})x_k \end{aligned}$$

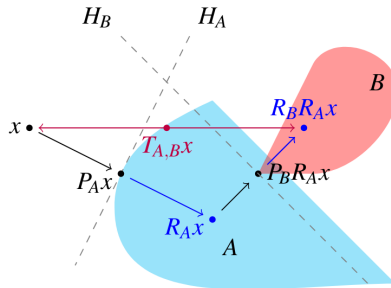
Cyclic, Dykstra , etc.

$$x_{k+1} = f(x_k)$$

Geometric Intuition



(a) One step of alternating projections



(b) One step of Douglas-Rachford method

Projection for the Bilinear constraint

Constraint:

$$C = \{(y, x, w) \mid y = w^\top x\}$$

Projection Objective: Find closest point $(y, x, w) \in \mathbb{R} \times \mathbb{R}^n \times \mathbb{R}^n$ to the prior estimate (y_0, x_0, w_0) (If $\gamma \rightarrow \infty$: $y = y_0$):

$$\underset{y, x, w}{\text{minimize}} \quad \|x - x_0\|^2 + \|w - w_0\|^2 + \gamma^2(y - y_0)^2$$

Projection for the Bilinear constraint

The Lagrangian with multiplier λ is:

$$L = \|x - x_0\|^2 + \|w - w_0\|^2 + \gamma^2(y - y_0)^2 + \lambda(y - w^\top x)$$

Setting $\nabla L = 0$ yields (with $u = \lambda/2$):

$$\begin{aligned}x &= x_0 + uw & y &= y_0 - \frac{u}{\gamma^2} \\w &= w_0 + ux & y &= w^\top x\end{aligned}$$

Solving the coupled system for x and w :

$$x = \frac{x_0 + uw_0}{1 - u^2}, \quad w = \frac{w_0 + ux_0}{1 - u^2}$$

Projection for the Bilinear constraint

Substituting $x(u)$, $w(u)$, $y(u)$ back into the constraint $y = w^\top x$:

$$\left(y_0 - \frac{u}{\gamma^2}\right) = \frac{(w_0 + ux_0)^\top (x_0 + uw_0)}{(1 - u^2)^2}$$

This simplifies to a scalar root-finding problem $h(u) = 0$:

$$h(u) = \frac{p(1 + u^2) + qu}{(1 - u^2)^2} - y_0 + \frac{u}{\gamma^2} = 0$$

where $p = w_0^\top x_0$ and $q = \|x_0\|^2 + \|w_0\|^2$.

Solution: Solve for u (e.g., via Newton's method), then reconstruct x , w , y .

Experiments

JAX optimizer

Experimental Setup

Goal: Evaluate the feasibility and efficiency of projection-based learning compared to standard baselines.

- **Datasets:**
 - LED for NMF.
 - Synthetic Binary Vectors for classification.
- **Metrics:** Convergence Rate (Error vs Iterations), Execution Time
- **Hardware:** All experiments run on Intel i5-1135G7 (CPU only).

Experiment 1: Binary Classification

- **Task:** Learn random mapping ($x \in \{0, 1\}^5 \rightarrow y \in \{0, 1\}$).
- **Architecture:**
 - 1 Hidden Layer (1024 units).

performance

AP+STEP		26.69s
AP+RELU		40.0s
ADAM		1.98s

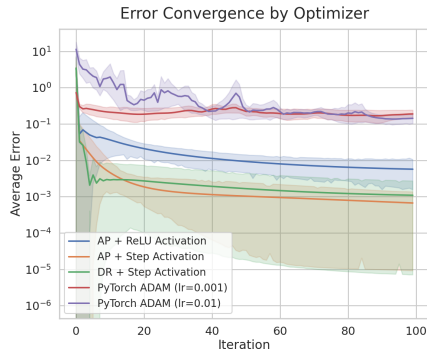


Figure: Convergence Rate

Experiment 2: Non-negative Matrix Factorization

- **Formal Definition:**

Given $V \in \mathbb{R}^{m \times n}$ and rank r :

$$\underset{W \in \mathbb{R}_{\geq 0}^{m \times r}, H \in \mathbb{R}_{\geq 0}^{r \times n}}{\text{minimize}} \quad \frac{1}{2} \|V - WH\|_F^2$$

- Matrix Size: 6×6
- Rank: 5

Algorithm	Error (MSE)	Time
Douglas-Rachford (LWL)	-	2s
Douglas-Rachford	3.41e-02	8s
scikit-learn (CD)	1.98e-02	<1s

Other Research Avenues (Status)

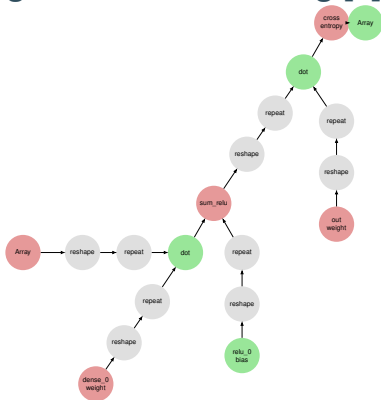
Topic	Outcome
Integer Weights (SAT Solver) Functional but slow. High hyperparameter sensitivity.	DONE
Autoencoder Unstable training across multiple batches.	DEBUGGING
Alternative Bilinear Constraint Converged to suboptimal local minima compared to standard projection.	VALIDATING
Regularization (Rank/Ortho) Did not improve convergence consistently.	VALIDATING
Testing Different Optimizers Variants being evaluated.	VALIDATING
Effect of Batch Size Multi-batch training degrades convergence; under investigation.	PLANNED
Gapped step activation Testing gapped step vs regular step.	RESEARCHING

PJAX

Projection-based framework

Overview

A Projection-based framework for gradient-free learning [2]



FEATURES

- **Graph based**
Elegant handling of layer connections and projection operators.
- **JAX-based Backend**
Compiles to XLA for efficient GPU execution.
- **Layer Library**
Implementations of Dense, CNN layers.

Graph Architecture

MODULE

High-level blocks (CNN, MLP).

Computation

e.g. ReLU, Linear

Ops & Projections

Core operations + Projections

Transforms

e.g. Reshape

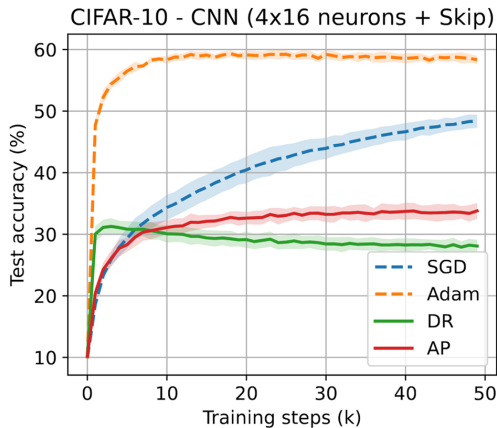
No-ops & Inverse

Reshape + Inverse

THE GRAPH

CNN Implementation Challenges

- **Slow convergence**
- **Memory Bottleneck**
“4-layer CNN necessitated reducing the number of hidden units per layer to 16 to fit within the available 96GB GPU memory”.



CIFAR-10 - CNN (Limited Scalability)

FFT-Based Convolution

FFT

1

Transform input and kernel
to frequency domain.

Hadamard

2

Element-wise product.

IFFT

3

Transform back.

Current Status: FFT-CNN

- **Complexity:** having many filters results in weak performance.
- **Current Limitation:** variable padding and stride are not supported.

Ongoing Work

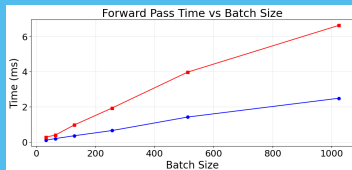


Figure: Scaling of forward

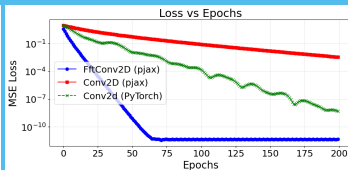


Figure: Performance on Synthetic benchmark

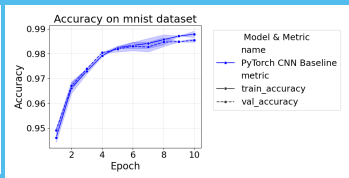


Figure: Performance on mnist

Ongoing PJAX Experiments

Topic	Status
Autoencoder (PJAX) Slow convergence compared to standard implementation.	DEBUGGING
MSE Loss Projection Projection-based MSE loss layer implemented.	DONE
Douglas–Rachford Variants Different DR formulations are being evaluated to study their effect on convergence.	VALIDATING
Alternative Optimization Strategy Exploring the effect of propagating updates through the computational graph rather than using strictly local projections.	RESEARCHING
Variational Autoencoder Work is ongoing to express the KL divergence term as a set of constraints suitable for projection-based optimization.	RESEARCHING
Diffusion Neural Network Initial testing indicates that PJAX's current performance limits make large convolution-based diffusion architectures impractical; further optimization is required..	ON HOLD

Timeline

30 JAN - 14 FEB

Optimizing and profiling PJAX

21 MAR - 28 MAR

Positional embeddings

19 FEB - 4 MAR

Diffusion neural net

29 MAR - 4 APR

Benchmarking and testing the transformer

8 MAR - 21 MAR

Transformer layer

16 APR -

Poster and writing

Questions ?